

Materi Ajar

Aplikasi Mobile Lanjut

Topik: Pembuatan Dashboard CRUD Data Mahasiswa Menggunakan Flutter dan Firebase

1. Deskripsi Mata Kuliah / Modul

Materi ini dirancang untuk mahasiswa **level menengah-lanjut** yang telah memahami dasar pemrograman mobile dan ingin mengembangkan aplikasi **production-ready** menggunakan **Flutter** sebagai frontend dan **Firestore** sebagai Backend-as-a-Service (BaaS).

Pendekatan pembelajaran menggunakan:

- ✓ Project-Based Learning
 - ✓ Praktikum Dominan (70%)
 - ✓ Studi Kasus Dunia Nyata
 - ✓ Outcome-Based Education (OBE)
-

2. Capaian Pembelajaran (Learning Outcomes)

Setelah mengikuti modul ini mahasiswa mampu:

1. Mendesain arsitektur aplikasi mobile berbasis cloud.
 2. Mengintegrasikan Flutter dengan Firestore.
 3. Mengimplementasikan autentikasi pengguna.
 4. Membangun dashboard interaktif.
 5. Mengembangkan fitur CRUD (Create, Read, Update, Delete).
 6. Mengelola state management.
 7. Menerapkan validasi form.
 8. Melakukan deployment aplikasi.
-

3. Prasyarat

Mahasiswa harus telah menguasai:

- Dasar Dart
 - Widget Flutter
 - OOP
 - REST API (basic understanding)
 - Database concept
-

4. Konsep Utama

4.1 Apa itu Flutter?

Framework UI open-source dari Google untuk membangun aplikasi **cross-platform** dari satu codebase.

Keunggulan:

- Hot reload
 - Performa mendekati native
 - UI konsisten
 - Produktivitas tinggi
-

4.2 Apa itu Firebase?

Platform cloud yang menyediakan layanan backend siap pakai:

Fitur utama:

- Authentication
 - Cloud Firestore
 - Realtime Database
 - Cloud Storage
 - Hosting
-

5. Arsitektur Sistem

Arsitektur yang Digunakan:

Client–Cloud Architecture

Flutter App → Firebase Authentication → Cloud Firestore

Alur Sistem:

1. User login
 2. Firebase verifikasi
 3. Dashboard tampil
 4. Data mahasiswa diambil dari Firestore
 5. User dapat melakukan CRUD
-

6. Persiapan Lingkungan Development

Install Tools

1. Flutter SDK

Cek instalasi:

```
flutter doctor
```

2. Android Studio / VS Code

Install extension:

- Flutter
 - Dart
-

6.2 Membuat Project Baru

```
flutter create dashboard_mahasiswa  
cd dashboard_mahasiswa
```

Jalankan:

```
flutter run
```

7. Integrasi Flutter dengan Firebase

Step 1 — Buat Project Firebase

1. Buka Firebase Console
 2. Klik **Add Project**
 3. Ikuti wizard
-

Step 2 — Tambahkan App Android

Gunakan package name:

```
com.example.dashboardmahasiswa
```

Download:

```
google-services.json
```

Letakkan pada:

```
android/app/
```

Step 3 — Install FlutterFire CLI

```
dart pub global activate flutterfire_cli
```

Konfigurasi:

```
flutterfire configure
```

Step 4 — Tambahkan Dependency

```
dependencies:  
  firebase_core: latest  
  firebase_auth: latest  
  cloud_firestore: latest
```

Step 5 — Inisialisasi Firebase

Pada main.dart:

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp();  
  runApp(MyApp());  
}
```

8. Struktur Project (Best Practice)

Gunakan arsitektur **Clean Architecture (simplified)**:

```
lib/  
├── models  
└── services
```

9. Membuat Model Mahasiswa

mahasiswa_model.dart

```
class Mahasiswa {
  String id;
  String nama;
  String nim;
  String jurusan;

  Mahasiswa({
    required this.id,
    required this.nama,
    required this.nim,
    required this.jurusan,
  });

  Map<String, dynamic> toMap() {
    return {
      'nama': nama,
      'nim': nim,
      'jurusan': jurusan,
    };
  }

  factory Mahasiswa.fromFirestore(Map<String, dynamic> data, String id) {
    return Mahasiswa(
      id: id,
      nama: data['nama'],
      nim: data['nim'],
      jurusan: data['jurusan'],
    );
  }
}
```

10. Service Firestore (CRUD Engine)

firestore_service.dart

```
class FirestoreService {
  final CollectionReference mahasiswa =
    FirebaseFirestore.instance.collection('mahasiswa');

  // CREATE
  Future<void> tambahMahasiswa(Mahasiswa mhs) {
    return mahasiswa.add(mhs.toMap());
  }

  // READ
```

```

Stream<List<Mahasiswa>> getMahasiswa() {
  return mahasiswa.snapshots().map((snapshot) =>
    snapshot.docs.map((doc) =>
      Mahasiswa.fromFirestore(
        doc.data() as Map<String, dynamic>, doc.id)).toList());
}

// UPDATE
Future<void> updateMahasiswa(Mahasiswa mhs) {
  return mahasiswa.doc(mhs.id).update(mhs.toMap());
}

// DELETE
Future<void> deleteMahasiswa(String id) {
  return mahasiswa.doc(id).delete();
}
}

```

11. Membuat Dashboard

Konsep Dashboard

Dashboard harus memiliki:

- ✓ AppBar
 - ✓ List Data
 - ✓ Floating Action Button
 - ✓ Navigation
 - ✓ Search (optional advanced)
-

dashboard_screen.dart

Gunakan **StreamBuilder** agar realtime.

```

StreamBuilder<List<Mahasiswa>>(
  stream: FirestoreService().getMahasiswa(),
  builder: (context, snapshot) {
    if (!snapshot.hasData) {
      return Center(child: CircularProgressIndicator());
    }

    final data = snapshot.data!;

    return ListView.builder(
      itemCount: data.length,
      itemBuilder: (context, index) {
        final mhs = data[index];

        return ListTile(
          title: Text(mhs.nama),

```

```

        subtitle: Text(mhs.nim),
        trailing: IconButton(
            icon: Icon(Icons.delete),
            onPressed: () {
                FirestoreService().deleteMahasiswa(mhs.id);
            },
        ),
    ),
);
},
);
},
)

```

12. Form Tambah Mahasiswa

Gunakan GlobalKey<FormState>

Field wajib:

- Nama
- NIM
- Jurusan

Validasi:

```

validator: (value) {
    if (value!.isEmpty) {
        return 'Tidak boleh kosong';
    }
    return null;
}

```

Simpan Data:

```

if (_formKey.currentState!.validate()) {
    FirestoreService().tambahMahasiswa(
        Mahasiswa(
            id: '',
            nama: namaController.text,
            nim: nimController.text,
            jurusan: jurusanController.text,
        ),
    );
}

```

13. Edit Data Mahasiswa

Strategi terbaik:

- 🔗 Reuse form tambah
- 🔗 Kirim object mahasiswa

```
FirestoreService().updateMahasiswa(mhs);
```

14. Hapus Data (Best Practice)

Gunakan dialog konfirmasi:

```
showDialog(  
  context: context,  
  builder: (_) => AlertDialog(  
    title: Text("Konfirmasi"),  
    content: Text("Hapus data mahasiswa?"),  
    actions: [  
      TextButton(  
        onPressed: () => Navigator.pop(context),  
        child: Text("Batal"),  
      ),  
      TextButton(  
        onPressed: () {  
          FirestoreService().deleteMahasiswa(id);  
          Navigator.pop(context);  
        },  
        child: Text("Hapus"),  
      ),  
    ],  
  ),  
);
```

15. Authentication (Highly Recommended)

Gunakan Firebase Auth:

- Email & Password
- Role-based access (Admin / User)

Flow:

```
Login → Firebase Auth → Dashboard
```

16. State Management (Recommended)

Untuk aplikasi lanjut gunakan:

- ☆ Provider (mudah)
- ☆ Riverpod (modern)
- ☆ Bloc (enterprise)

Rekomendasi pengajaran: **Provider** → **Riverpod**

17. Keamanan Firestore

Rules sederhana:

```
allow read, write: if request.auth != null;
```

Artinya hanya user login yang dapat akses.

18. Pengujian Aplikasi

Lakukan:

- ☒ Unit Testing
 - ☒ Widget Testing
 - ☒ Integration Testing
-

19. Deployment

Build APK:

```
flutter build apk
```

Atau:

```
flutter build appbundle
```

Upload ke Play Console.

20. Mini Project Mahasiswa

 **Project:**

Sistem Informasi Akademik Mobile

Fitur wajib:

- Login
- Dashboard
- CRUD mahasiswa
- Search
- Logout

Fitur nilai A:

- ☆ Pagination
 - ☆ Dark Mode
 - ☆ Export PDF
 - ☆ Role Admin
-

21. Rubrik Penilaian (OBE Friendly)

Komponen	Bobot
UI/UX	20%
Integrasi Firebase	20%
CRUD berjalan	25%
Clean Code	15%
Presentasi	10%
Inovasi	10%

22. Kesalahan yang Sering Terjadi

- ✗ Tidak menunggu Firebase initialize
 - ✗ Salah rules Firestore
 - ✗ Tidak validasi form
 - ✗ Struktur project berantakan
 - ✗ Tidak handle null
-

23. Best Practice Industri

- ✓ Gunakan MVVM / Clean Architecture
 - ✓ Pisahkan UI & logic
 - ✓ Gunakan repository pattern
 - ✓ Hindari hardcode
 - ✓ Logging penting
-

24. Latihan Bertahap

Beginner

- Tampilkan data Firestore

Intermediate

- Tambah + Edit

Advanced

- Full dashboard + auth + role
-

25. Penutup

Dengan menguasai modul ini mahasiswa telah berada pada **level siap kerja sebagai Mobile Developer Junior–Intermediate**, karena sudah memahami:

- ✓ Cloud backend
- ✓ Arsitektur aplikasi
- ✓ Realtime database
- ✓ Production workflow